TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Jorge Zambrano.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Gilver Valentin Macias Pilligua

PARALELO: "A"

SEMESTRE: 5

ACTIVIDAD: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2026 - ENERO 2026

Actividades a desarrollar en la Unidad _1___

Actividad # ____1____

Actividad #1: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal
Evidencias

1. Análisis del problema — Sistema SIGRAA

SIGRAA: Sistema de Gestión de Rescate y Adopción Animal

Asignatura: Desarrollo de Sistemas Informáticos · Unidad 1: Introducción y paradigmas de diseño de software

1.1. Descripción del problema

Los refugios de animales necesitan administrar los reportes de rescate, clasificar las especies, asignar responsables y realizar un seguimiento de cada animal. Cuando estos procesos se llevan a cabo mediante registros manuales o información dispersa, puede resultar difícil conocer el estado de los casos, identificar a los responsables y mantener un historial organizado.

Para solucionar esta problemática, se propone SIGRAA, un sistema que permitirá centralizar la información de los casos de rescate y facilitar la coordinación entre ciudadanos, voluntarios, veterinarios y administradores del refugio.

El sistema estará orientado a mejorar la organización de los reportes, el seguimiento de los animales rescatados y la gestión de las actividades relacionadas con su atención y posible adopción.

1.2. Identificación de los actores principales

Un actor es una persona o un rol externo que interactúa con el sistema para realizar determinadas actividades.

Actor 1. Ciudadano / Reportante

Responsabilidad: comunicar la existencia de un animal que necesita ayuda.

- Crear un caso de rescate.
- Registrar la ubicación y descripción del animal.
- Consultar el estado de un reporte.

Actor 2. Voluntario

Responsabilidad: colaborar en las actividades de rescate y seguimiento.

- Consultar los casos asignados.
- Registrar las acciones realizadas.
- Informar sobre el progreso de las actividades.

Actor 3. Veterinario

Responsabilidad: evaluar y atender la condición de salud de los animales.

- Consultar los casos asignados.
- Registrar diagnósticos y tratamientos.
- Actualizar el historial de atención veterinaria.

Actor 4. Administrador del refugio

Responsabilidad: coordinar y supervisar las operaciones del sistema.

- Clasificar los casos según la especie.
- Asignar voluntarios y veterinarios.
- Supervisar los estados de los casos.
- Administrar usuarios y gestionar los procesos de adopción.

Nota de modelado: la rúbrica menciona «Voluntario/Veterinario» en conjunto, pero conviene representarlos como actores separados porque realizan actividades diferentes. En el diagrama UML podrán compartir algunas interacciones si corresponde.

1.3. Requerimientos funcionales

Los requerimientos funcionales describen las acciones que SIGRAA debe permitir realizar.

Código	Requerimiento	Descripción
RF01	Crear caso de rescate	El sistema permitirá registrar un nuevo reporte de rescate.
RF02	Registrar datos del caso	Permitirá almacenar la ubicación, fecha, descripción y datos disponibles del animal.
RF03	Categorizar especie	Permitirá clasificar al animal como perro, gato o fauna silvestre.
RF04	Asignar voluntario	Permitirá al administrador asignar un voluntario responsable del caso.
RF05	Asignar veterinario	Permitirá asignar un veterinario cuando el caso requiera atención profesional.
RF06	Actualizar estado	Permitirá registrar el avance del caso durante el proceso de rescate.

RF07	Registrar atención	Permitirá documentar diagnósticos, tratamientos y observaciones veterinarias.
RF08	Consultar casos	Permitirá buscar reportes y consultar su estado actual.
RF09	Consultar historial	Permitirá revisar las actividades y modificaciones registradas en un caso.
RF10	Gestionar adopciones	Permitirá registrar, evaluar y dar seguimiento a las solicitudes de adopción.
RF11	Administrar usuarios	Permitirá gestionar las cuentas, los roles y los permisos de acceso.

Los tres requerimientos expresamente destacados en la rúbrica —crear casos, categorizar especies y asignar voluntarios— quedan incluidos como RF01, RF03 y RF04.

1.4. Requerimientos no funcionales

Los requerimientos no funcionales describen las características de calidad que debe tener el sistema.

Código	Requerimiento	Descripción
RNF01	Escalabilidad	La arquitectura deberá permitir incorporar nuevos tipos de animales, usuarios y funcionalidades sin rediseñar todo el sistema.
RNF02	Mantenibilidad	La organización del diseño deberá facilitar la comprensión, modificación y ampliación de sus componentes.
RNF03	Trazabilidad de versiones	Los cambios del diseño deberán registrarse mediante Git y GitHub, conservando el historial de commits y ramas.
RNF04	Seguridad	El acceso a las operaciones deberá depender del rol y los permisos del usuario.
RNF05	Integridad	El sistema deberá mantener consistencia en los datos de los casos, sus responsables y sus estados.
RNF06	Reutilización	El diseño deberá favorecer el uso de componentes comunes en distintas funcionalidades.

1.5. Reglas básicas del negocio

Para completar el análisis, se establecen estas reglas iniciales:

1. Cada caso de rescate tendrá un identificador único.
2. Un reporte deberá contener la información disponible sobre el animal y su ubicación.
3. La clasificación de especies será obligatoria cuando se disponga de información suficiente; los casos desconocidos deberán poder registrarse como pendientes de identificación.

4. La asignación de voluntarios y veterinarios corresponderá al administrador o al personal autorizado.
5. Los cambios importantes de estado deberán quedar registrados en el historial.
6. La atención de fauna silvestre deberá considerar las necesidades específicas de esa especie.
7. Una solicitud de adopción deberá evaluarse antes de su aprobación.

1.6. Resultado del análisis

El análisis de SIGRAA identifica cuatro actores principales y establece los requerimientos funcionales y no funcionales necesarios para organizar la gestión de rescates animales. También define reglas básicas que servirán como fundamento para el diseño de los diagramas UML, la selección de patrones de diseño y la arquitectura general del sistema.

Esta etapa establece qué debe hacer el sistema y qué características debe cumplir, sin entrar todavía en su programación.

2. Diseño de la arquitectura de software

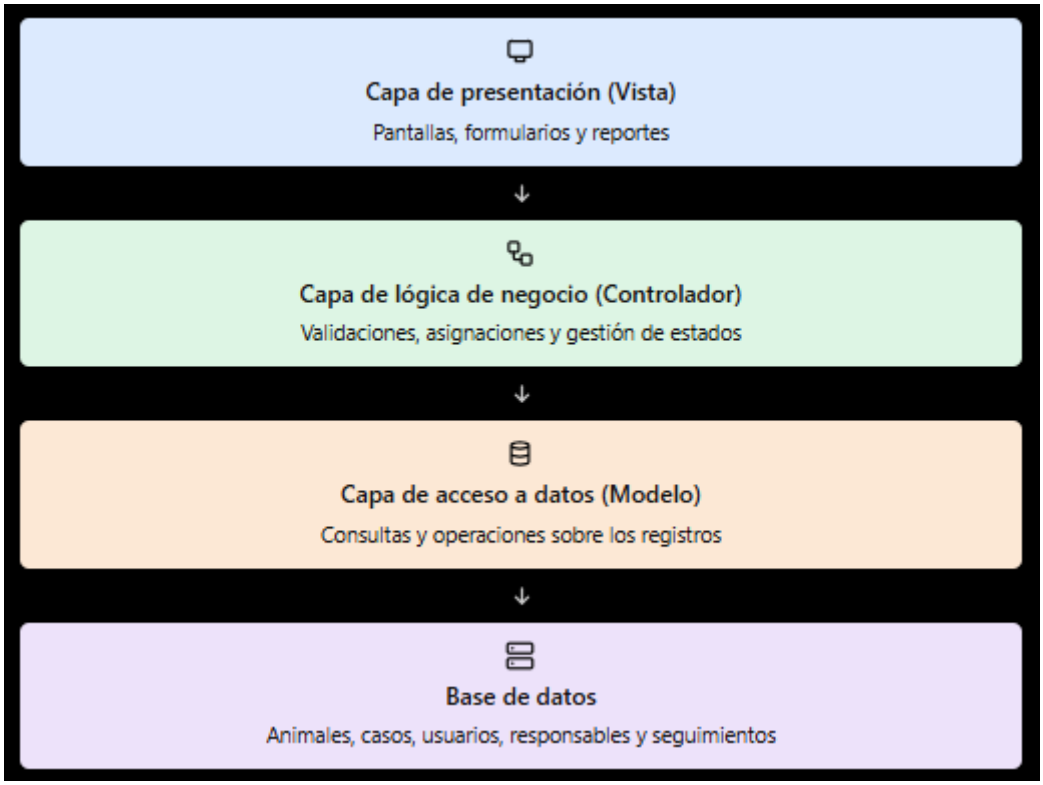
2.1. Arquitectura seleccionada

Para el desarrollo del sistema SIGRAA se propone una arquitectura en capas, utilizando el patrón MVC (Modelo-Vista-Controlador). Esta arquitectura permite organizar el sistema en componentes independientes, facilitando su mantenimiento, comprensión y futuras modificaciones.

La arquitectura se divide en cuatro capas principales:

- Presentación: permite que los usuarios interactúen con el sistema mediante formularios y pantallas.
- Lógica de negocio: procesa las operaciones, valida los datos y aplica las reglas del refugio.
- Acceso a datos: gestiona las consultas, inserciones, actualizaciones y eliminaciones de información.
- Base de datos: almacena los registros de animales, usuarios, responsables y casos de rescate.

2.2. Diagrama de arquitectura



El funcionamiento es sencillo: el usuario registra un animal desde la interfaz, el sistema valida la información, la capa de acceso a datos guarda el registro y la base de datos conserva la información para futuras consultas.

2.3. Patrones de diseño propuestos

Para mejorar la organización del sistema, se proponen los siguientes patrones de diseño:

MVC (Modelo-Vista-Controlador)

Separa la interfaz gráfica, la lógica de control y los datos. Por ejemplo, permite modificar la pantalla de registro de animales sin tener que cambiar toda la lógica del sistema.

Strategy (Estrategia)

Permite utilizar diferentes procedimientos según el tipo de animal o las necesidades del caso. Por ejemplo, aplicar criterios distintos para gestionar la atención de un perro rescatado y de un animal silvestre.

State (Estado)

Permite controlar las transiciones de cada caso. Por ejemplo, un animal puede pasar de «Rescatado» a «En tratamiento» y posteriormente a «En recuperación».

Observer (Observador)

Permite notificar a los usuarios responsables cuando se produce un cambio importante, como una nueva asignación veterinaria o una actualización del estado del caso.

2.4. Justificación de la arquitectura

La arquitectura en capas fue seleccionada porque permite separar las responsabilidades del sistema SIGRAA y facilita su mantenimiento. Esta organización ayuda a que las modificaciones realizadas en la interfaz no afecten directamente a la base de datos y permite agregar nuevas funcionalidades sin tener que reconstruir todo el sistema.

Además, la aplicación de patrones de diseño permite gestionar los estados de los casos, organizar los procedimientos de atención y comunicar los cambios importantes a los usuarios. De esta manera, se busca obtener un sistema organizado, escalable y adaptable a las necesidades del refugio animal.

2.5. Resultado esperado

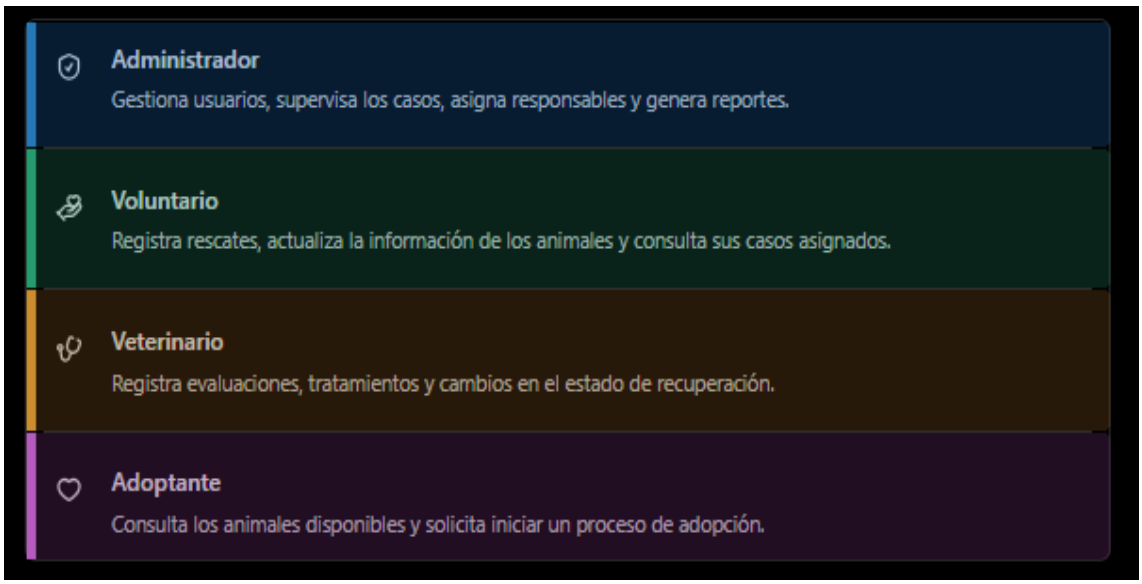
Con esta propuesta, SIGRAA contará con una estructura lógica para gestionar los registros de animales rescatados, asignar responsables, realizar seguimientos y controlar los procesos de atención y adopción.

Importante: este diseño representa una propuesta arquitectónica; los patrones indicados todavía deben reflejarse en los diagramas y documentos correspondientes si así lo exige la rúbrica.

2.1. Diagrama UML de casos de uso

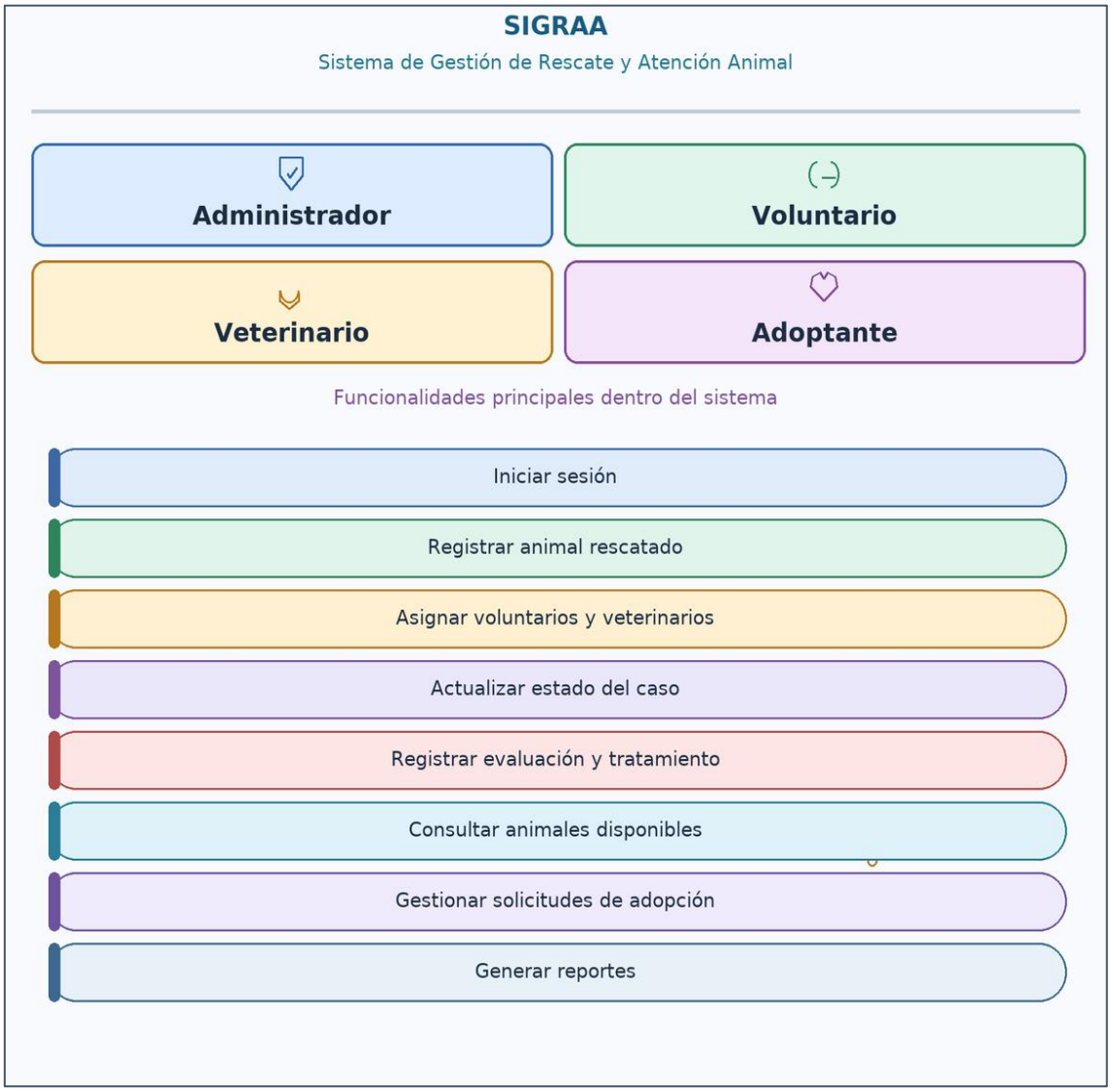
El diagrama de casos de uso permite representar quiénes utilizarán el sistema y qué acciones podrán realizar. En nuestro proyecto, los actores principales son el administrador, el voluntario, el veterinario y el adoptante

1. Actores del sistema

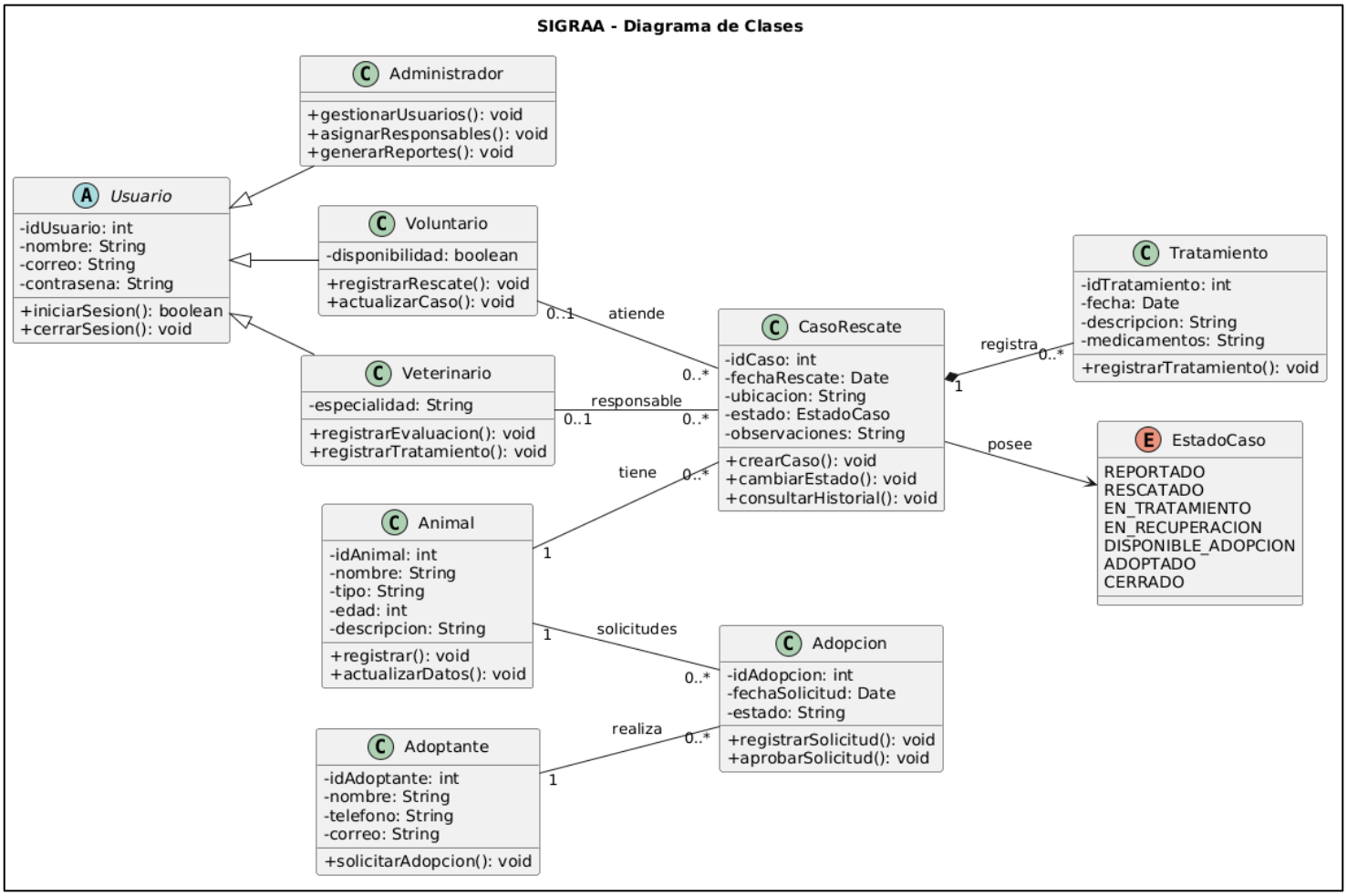


2. Diagrama de casos de uso de SIGRAA

Este es el contenido del diagrama que representaremos. El rectángulo delimita el sistema y los óvalos representan las funcionalidades.



2.1 Diagrama de clases UML del SIGRAA.

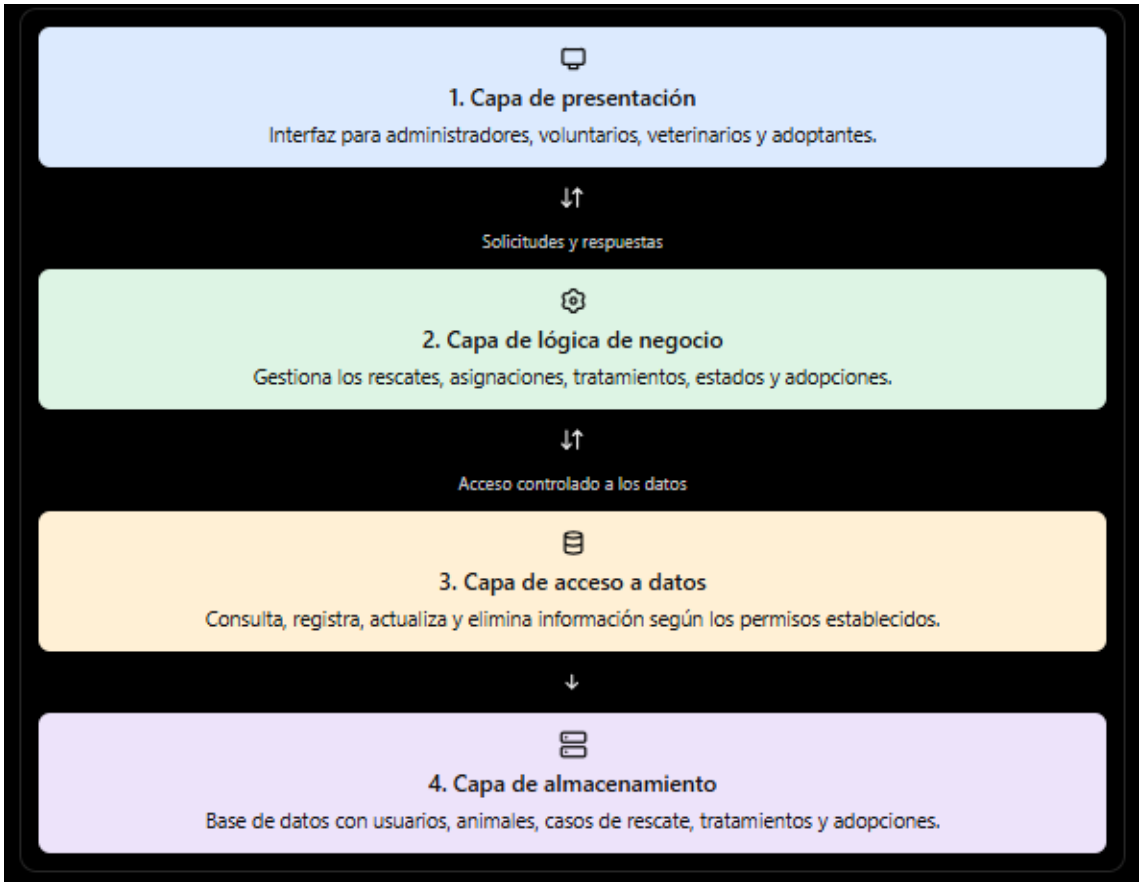


3. Diseño de la arquitectura general del sistema integrando el flujo de versiones

3.1. Arquitectura general del sistema

El sistema SIGRAA utilizará una arquitectura por capas, que permitirá organizar sus componentes de manera ordenada, facilitar el mantenimiento y mejorar la seguridad de la información.

La arquitectura estará compuesta por cuatro capas principales:



3.2. Funcionamiento de la arquitectura

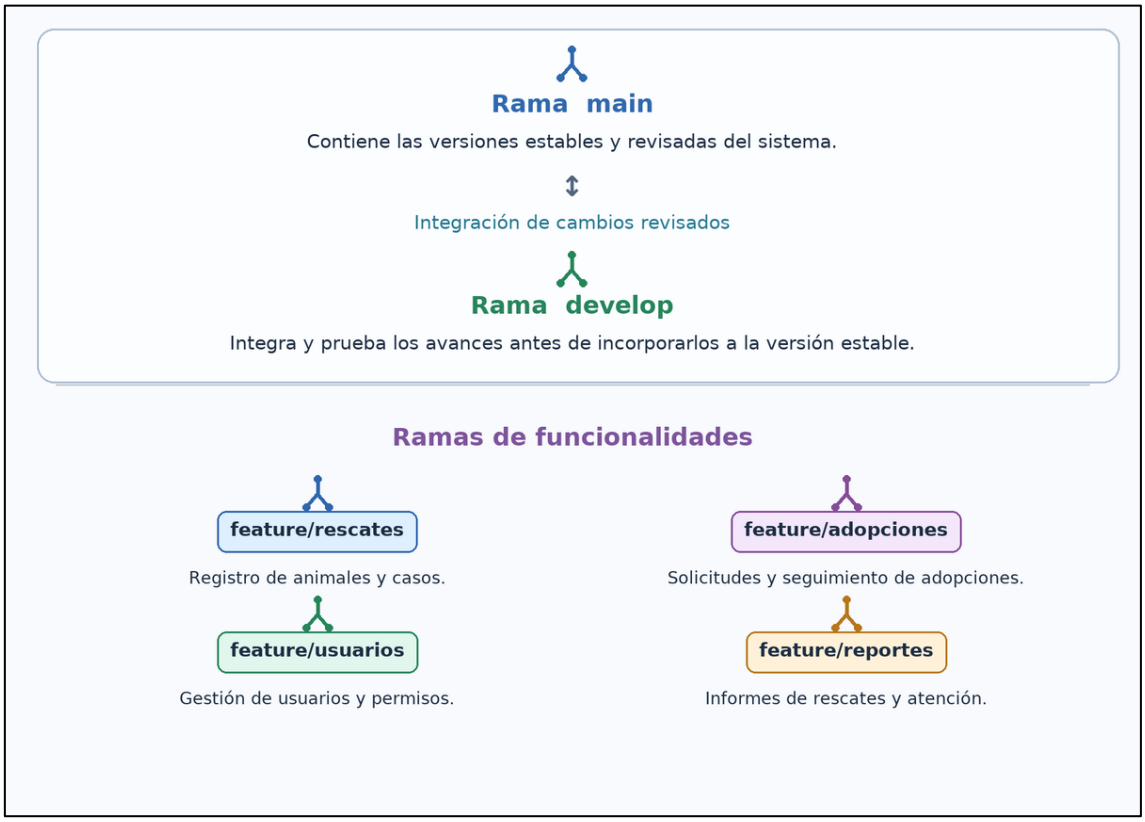
El sistema funcionará de la siguiente manera:

1. Un usuario ingresa al sistema mediante la interfaz.
2. La capa de presentación envía la solicitud a la lógica de negocio.
3. La lógica de negocio valida la información y los permisos del usuario.
4. La capa de acceso a datos realiza las operaciones necesarias en la base de datos.
5. El resultado regresa a la interfaz para que el usuario pueda visualizarlo.

Por ejemplo, cuando un voluntario registra un animal rescatado, el sistema almacena sus datos, crea el caso correspondiente y permite actualizar su estado durante el proceso de atención.

3.3. Integración del flujo de versiones

Para controlar los cambios realizados durante el desarrollo de SIGRAA se utilizará Git para el control de versiones y GitHub para alojar el repositorio remoto. Se propone trabajar con ramas para separar las funcionalidades nuevas del código estable.



Flujo de trabajo propuesto:

1. Crear una rama `feature/*` desde `develop` para trabajar en una funcionalidad.
2. Realizar cambios y guardarlos mediante commits descriptivos.
3. Enviar la rama al repositorio remoto de GitHub.
4. Crear una solicitud de integración (Pull Request) hacia `develop`.
5. Revisar y probar los cambios antes de integrarlos.
6. Cuando el sistema esté validado, integrar `develop` en `main` para publicar una versión estable.

Nota: `main` es el nombre habitual de la rama principal en GitHub. Si la rúbrica exige específicamente `master`, podemos adaptar el flujo a ese nombre.

3.4. Beneficios de integrar la arquitectura con el control de versiones

- Organización: cada capa tiene una responsabilidad definida.
- Mantenimiento: los cambios pueden realizarse sin mezclar todas las funcionalidades.
- Trazabilidad: los commits permiten identificar qué se modificó y cuándo.
- Trabajo colaborativo: las ramas permiten desarrollar funcionalidades por separado.

- Control de calidad: los cambios se revisan antes de incorporarse a la versión estable.
- Recuperación: el historial de Git facilita volver a una versión anterior cuando sea necesario.

3.5. Resultado esperado

La arquitectura general de SIGRAA permitirá gestionar los procesos de rescate, atención veterinaria y adopción mediante componentes organizados. La integración del flujo de versiones permitirá mantener un historial de cambios, desarrollar funcionalidades de forma independiente y controlar la incorporación de nuevas versiones del sistema.

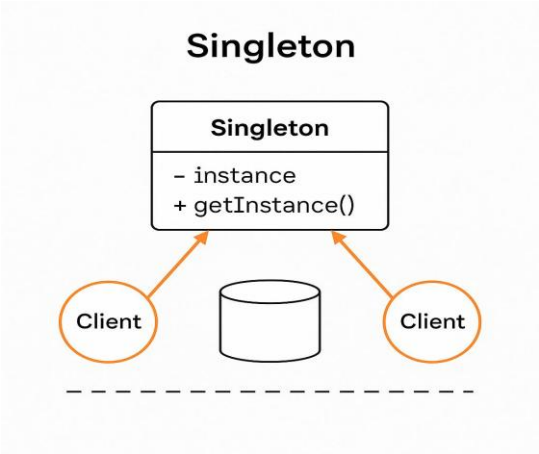
5. Aplicación de patrones de diseño

5.1. Patrones seleccionados

Utilizaremos cuatro patrones recomendados en la rúbrica: Singleton, Observer, Factory Method y MVC. De esta manera cumpliremos el mínimo de tres y cubriremos problemas diferentes del sistema.

5.2. Propósito y aplicación de cada patrón

1. Singleton — instancia central



Creación

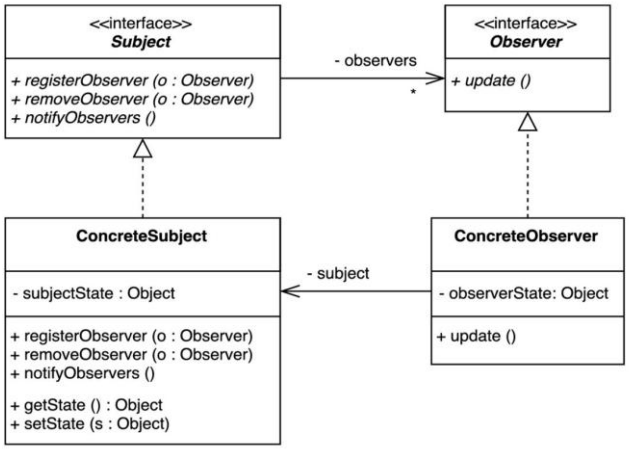
Propósito: garantizar que exista una única instancia de un componente central durante la ejecución de la aplicación.

Aplicación en SIGRAA: se utilizará en `GestorCasos`, responsable de coordinar operaciones generales de los casos de rescate.

Problema que resuelve: evita crear varias instancias del gestor central que puedan mantener estados inconsistentes. La conexión a la base de datos puede administrarse mediante un componente dedicado, según la tecnología utilizada.

Ejemplo: cuando se registra un rescate, los módulos que necesiten al gestor central acceden a la misma instancia.

2. Observer — notificaciones automáticas



Comportamiento

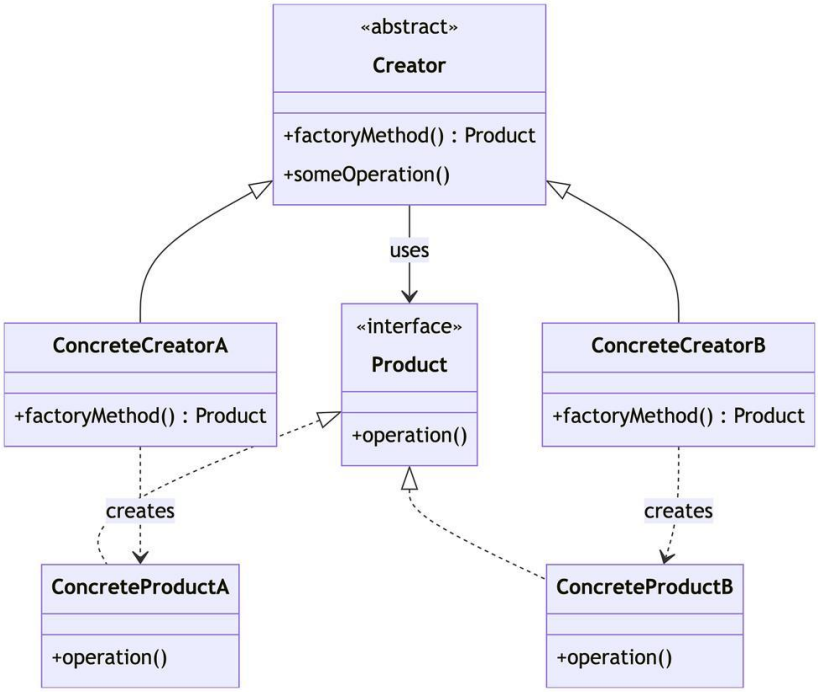
Propósito: notificar automáticamente a los componentes suscritos cuando ocurre un evento.

Aplicación en SIGRAA: los voluntarios suscritos a las alertas recibirán una notificación cuando se registre un nuevo rescate.

Problema que resuelve: evita que el módulo de rescates tenga que conocer individualmente a todos los destinatarios de cada notificación.

Ejemplo: al registrarse un animal abandonado, el sistema genera un evento y notifica a los voluntarios correspondientes.

3. Factory Method — creación de expedientes



Creación

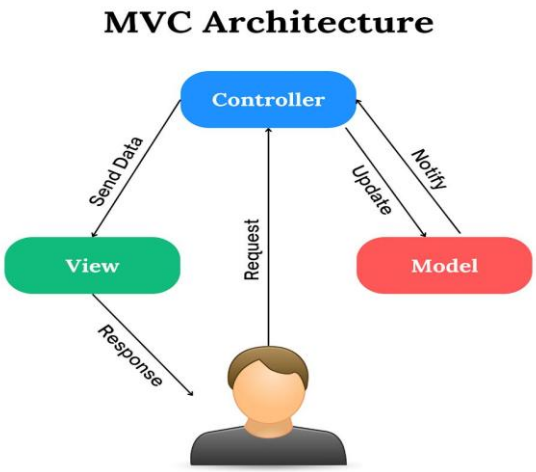
Propósito: separar la creación de objetos de la lógica que los utiliza.

Aplicación en SIGRAA: `FabricaExpedientes` permitirá crear expedientes especializados para perros, gatos y fauna silvestre.

Problema que resuelve: evita concentrar todas las condiciones de creación en una sola clase y facilita incorporar nuevas especies.

Ejemplo: al registrar un animal, la fábrica crea un `ExpedientePerro`, `ExpedienteGato` o `ExpedienteFaunaSilvestre`, según corresponda.

4. MVC — organización del sistema



Arquitectura

Propósito: separar los datos, la interfaz y el control de las solicitudes.

Aplicación en SIGRAA:

- Modelo: representa animales, casos, tratamientos y adopciones.
- Vista: muestra formularios, listados y estados de los rescates.
- Controlador: recibe las acciones del usuario y coordina las operaciones.

Problema que resuelve: evita mezclar las reglas del negocio con la interfaz gráfica, haciendo más sencillo mantener y probar el sistema.

Propósito de los patrones de diseño utilizados en SIGRAA

En el sistema SIGRAA, que está diseñado para gestionar los casos de rescate animal, se utilizan tres patrones de diseño: Singleton, Observer y Factory Method. Cada uno cumple una función importante y ayuda a resolver problemas que podrían presentarse durante el desarrollo y funcionamiento del sistema.

1. Patrón Singleton

El patrón Singleton permite que una clase tenga una sola instancia durante la ejecución del sistema.

En SIGRAA, se utiliza para centralizar la gestión de los casos de rescate. De esta manera, se evita crear varios gestores que puedan provocar desorganización o inconsistencias en los registros.

Ejemplo: cuando un voluntario registra un perro rescatado, el gestor central se encarga de coordinar el registro de la información y mantener el control del caso.

2. Patrón Observer

El patrón Observer permite que el sistema notifique automáticamente a otros componentes cuando ocurre un evento importante.

En SIGRAA, se utiliza para informar a los voluntarios o veterinarios cuando se registra un nuevo animal rescatado. Así, no es necesario revisar constantemente el sistema para conocer los nuevos casos.

Ejemplo: si se registra un gato que necesita atención veterinaria, el sistema puede enviar una notificación a los responsables correspondientes para que conozcan el caso y puedan actuar oportunamente.

3. Patrón Factory Method

El patrón Factory Method permite crear objetos sin que el código principal tenga que encargarse directamente de todos los detalles de su creación.

En SIGRAA, se utiliza para generar expedientes de acuerdo con la especie del animal, como perros, gatos o fauna silvestre. Cada tipo de expediente puede tener características específicas según las necesidades del animal.

Ejemplo: cuando se registra un animal de fauna silvestre, el sistema crea el expediente correspondiente sin necesidad de modificar todo el proceso general de registro.

Conclusión

La implementación de estos tres patrones ayuda a que SIGRAA sea más organizado, flexible y fácil de mantener. Singleton permite centralizar la gestión de los casos, Observer facilita las notificaciones automáticas y Factory Method simplifica la creación de expedientes según la especie. De esta manera, el sistema puede responder mejor a las necesidades del refugio y facilitar futuras mejoras sin tener que modificar toda su estructura.

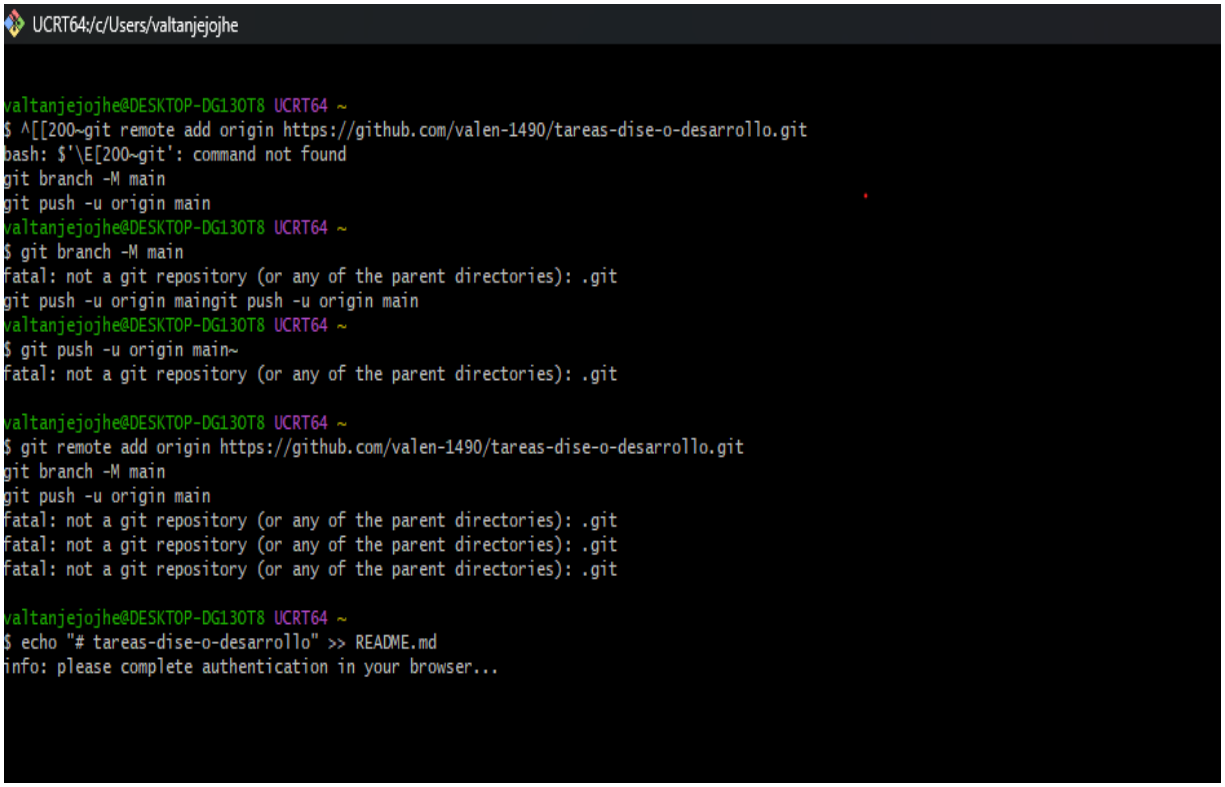
Justificación del uso de Gitflow en el proyecto SIGRAA

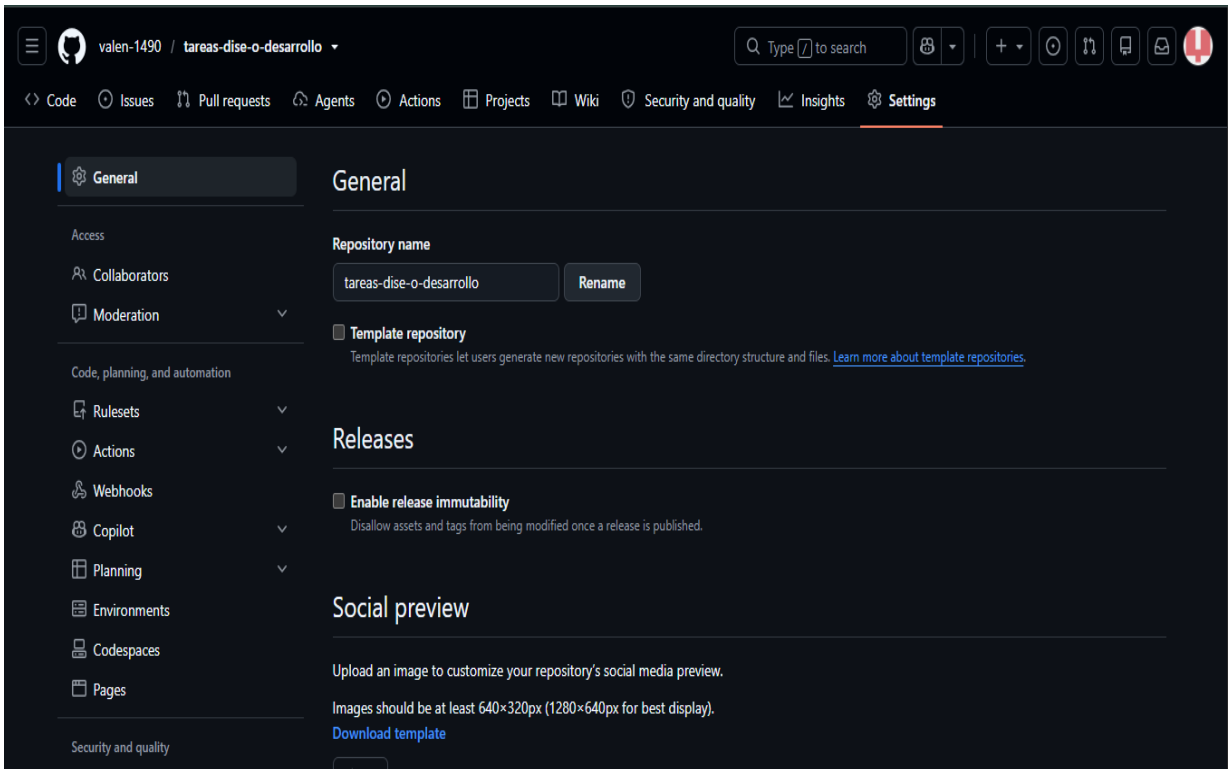
1. ¿Por qué utilizamos Gitflow?

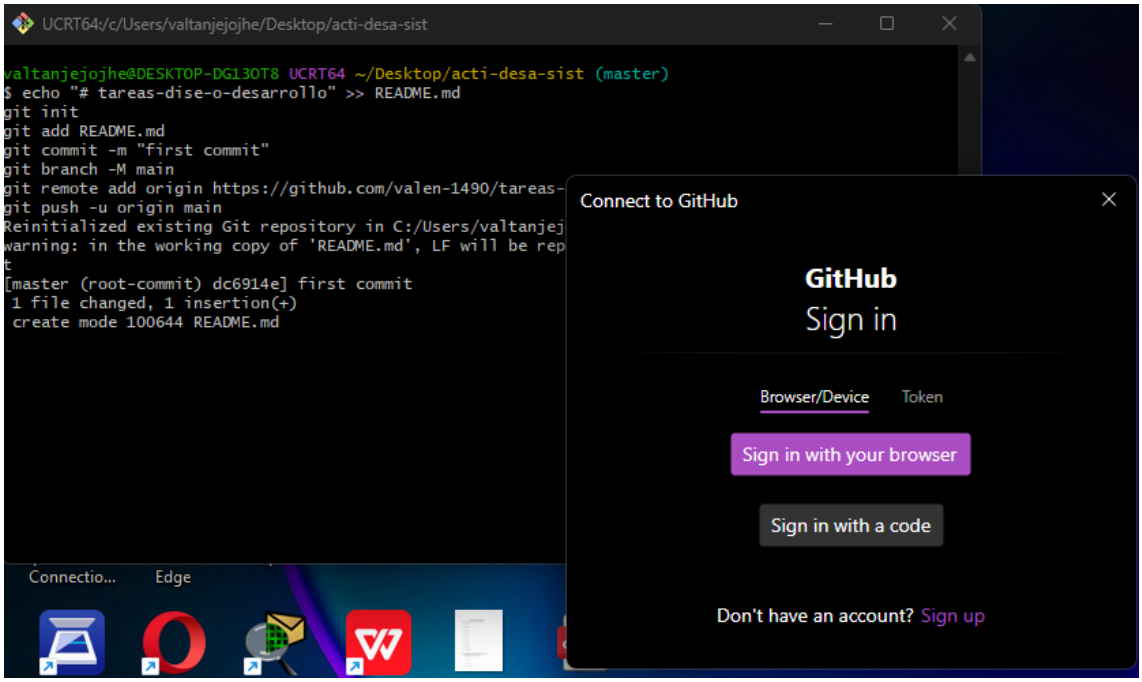
Gitflow es una estrategia de trabajo con Git que permite organizar el desarrollo del software mediante ramas. En el proyecto SIGRAA (Sistema de Gestión de Rescates y Atención Animal), su utilización ayuda a mantener el código organizado, separar las nuevas funcionalidades de la versión estable y controlar los cambios realizados durante el desarrollo.

Este sistema necesita funcionalidades como el registro de animales rescatados, la asignación de voluntarios y veterinarios, la gestión de expedientes y las notificaciones automáticas. Trabajar con Gitflow permite desarrollar estas funcionalidades de forma independiente y después integrarlas al proyecto principal.

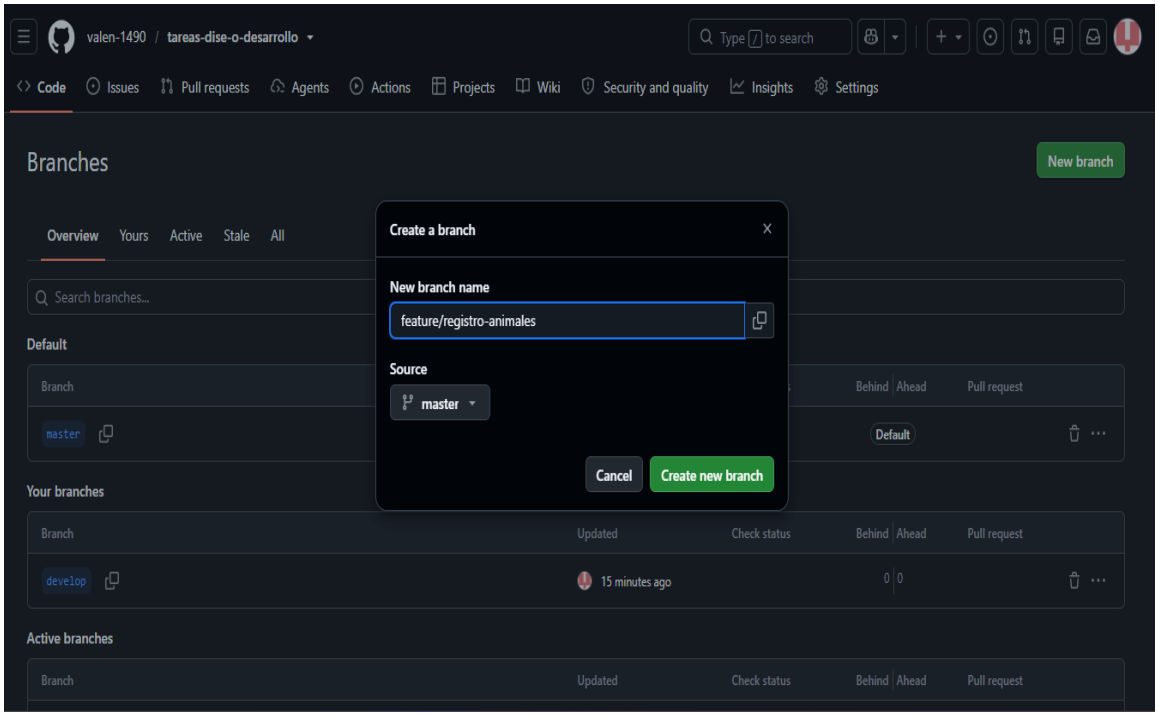
Captura de pantalla de la creación del repositorio

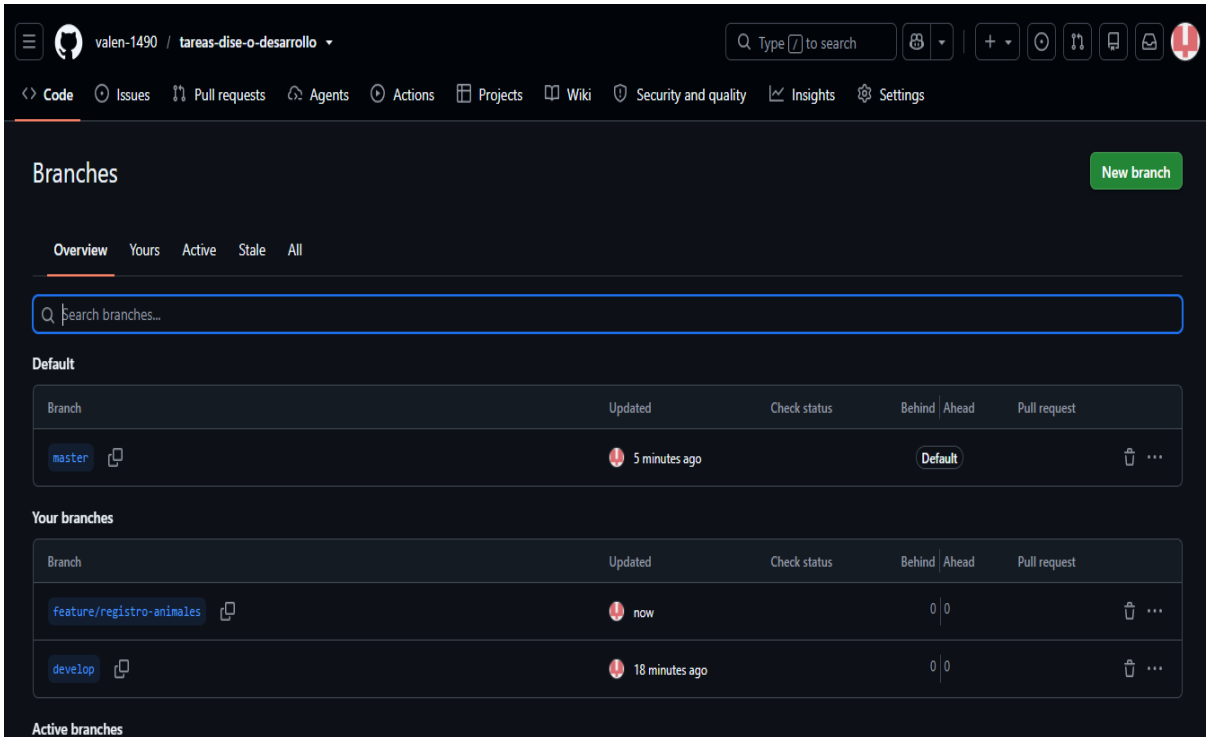






Creación y configuración de las ramas





https://github.com/valen-1490/tareas-dise-o-desarrollo/settings

enlace de GitHub

Bibliografía

(Referencias bibliográficas y/o enlaces en Internet (normas APA) que se utilizaron para realizar la práctica.)